AI-ASSISTED PREREQUISITE LEARNING PATH APPLICATION

Design and Analysis of Algorithms (DAA) Project

Project purpose

Build an application that accepts a student's list of courses, identifies plausible prerequisite relationships with help from a curated knowledge base and an AI API, and uses graph algorithms to produce an explainable learning order.

Core principle

The AI proposes educational relationships; the knowledge base supplies reviewed context; validation rules flag uncertain or conflicting relationships; and DAA algorithms process the resulting graph. A mathematically valid graph does not automatically prove that every educational relationship is correct.

Shared architecture guide for all project team members

Version 1.0 | 28 October 2026

Contents

1. Project overview and problem statement
2. What the user sees
3. Complete system flow
4. The application's brain: knowledge base, AI API, and validation
5. Representing prerequisites as a graph
6. DAA concepts and their role
7. Technology stack and system modules
8. Example walkthrough
9. Evaluation, testing, and success criteria
10. Limitations, risks, and responsible design
11. Step-by-step implementation roadmap
12. Key terms and viva-ready summary

Audience: all team members, including those new to graph algorithms or AI integration.

1. Project Overview and Problem Statement

Working name: AI-Assisted Prerequisite Learning Path Application (Dependency-Aware Course Registration Assistant — Intelligent Edition).

Students often know which subjects they want to learn but do not know the order in which to study them. A student interested in AI may enter Python Programming, Linear Algebra, Machine Learning, and Deep Learning without knowing which subjects are useful foundations for others.

The application accepts course names and an optional learning goal, such as AI Engineer, Data Scientist, Software Developer, or general Computer Science. It suggests prerequisite relationships, shows them as a directed graph, and uses graph algorithms to produce a valid learning order.

What the project is — and is not

- **It is:** an AI-assisted course-dependency analyzer with an explicit DAA algorithm engine.
- **It is not:** a general AI tutor that teaches course content.
- **It is not:** a system that guarantees one universally best learning order for every student.
- **It does not initially train a new ML model:** the first version uses an existing AI model through an API.

Main objectives

- Infer plausible prerequisite relationships from a user's selected courses.
- Explain why a relationship is suggested and distinguish formal requirements from recommendations.
- Represent accepted relationships as a directed graph.
- Generate a valid course order using topological sorting.
- Detect cycles and flag conflicting dependencies.
- Demonstrate algorithm correctness and efficiency through testing and performance analysis.

Scope boundary: the system generates a learning order based on the relationships it accepts. It should describe the result as a valid, explainable recommendation—not a guaranteed optimal curriculum.

2. What the User Sees

The user-facing workflow should be simple. The complexity belongs in the backend, not in the user's hands.

Example input form

Field	Example
Course names	Python Programming; Linear Algebra; Machine Learning; Deep Learning
Learning goal (optional)	AI Engineer
Action	Generate Learning Path

After clicking Generate, the user should see four main areas:

Screen/area	What it shows
1. Generate Learning Path	Course input, learning goal, validation messages, and Generate button.
2. Dependency Graph	Interactive graph with courses as nodes and prerequisite relationships as directed arrows.
3. Recommended Learning Order	A sequence that respects accepted dependency edges, plus explanations and warnings.
4. Algorithm Analysis Dashboard	Node/edge counts, cycle status, algorithm outputs, test results, and performance comparisons.

Users should be able to inspect an edge, read its explanation, and review uncertain relationships. If a user changes or rejects a relationship, the graph and learning order should be regenerated.

3. Complete System Flow

The application is divided into stages. Each stage has one job and passes a structured result to the next stage.

Stage	What happens	Output
1. User input	User enters course names and optionally a learning goal.	Raw course list and goal
2. Input cleanup	Remove duplicates, standardize aliases, identify ambiguous or unknown names.	Normalized course list
3. Knowledge-base lookup	Retrieve course descriptions and reviewed relationships already stored.	Known course information
4. AI analysis	Ask an LLM API to propose relevant relationships and explanations.	Structured relationship suggestions
5. Validation	Check endpoints, relationship type, evidence, uncertainty, and contradictions.	Accepted and flagged relationships
6. Graph construction	Convert courses to nodes and accepted relationships to directed edges.	Directed graph / adjacency list
7. DAA engine	Run topological sorting, cycle detection, and graph checks.	Ordering and graph diagnostics
8. Results display	Show graph, order, explanations, warnings, and algorithm analysis.	User-facing results

Flow in one line

Course names + goal → normalization → knowledge base → AI suggestions → validation → directed graph → DAA algorithms → learning order and dashboard

Why the stages are separate

If the learning order is wrong, the team must be able to determine whether the problem came from the AI suggestion, knowledge base, validation rules, or graph algorithm. Separate stages make the system easier to debug, test, explain, and improve.

4. The Application's Brain

The 'brain' is not one custom-trained model. It is a pipeline combining stored educational information, an existing AI model, and rules that check the proposed relationships.

4.1 Knowledge base: the application's reference memory

The first version will use a JSON file, for example `course_knowledge_base.json`. It will store course names, aliases, categories, descriptions, and reviewed relationships. JSON is readable and simple to maintain; a database is not necessary for the first prototype.

A relationship record should include at least:

- **from:** the earlier/foundation course.
- **to:** the course that may benefit from that foundation.
- **type:** required, recommended, or optional supporting knowledge.
- **reason:** a short explanation of the relationship.
- **source/status:** where it came from and whether it has been reviewed.

Start with a manageable set of commonly used Computer Science and AI courses. Review initial relationships using credible course descriptions or curricula rather than treating every example in this guide as an established fact.

Illustrative relationship record

```
{
  "from": "Machine Learning",
  "to": "Deep Learning",
  "type": "recommended",
  "reason": "Machine Learning concepts can help explain deep learning methods.",
  "status": "reviewed"
}
```

4.2 AI API: suggesting relationships

An API lets our backend send a carefully worded request to an existing AI model and receive a response. The prompt will include the user's courses, learning goal, relevant knowledge-base information, and instructions to avoid inventing relationships or calling every relationship mandatory.

The AI should return structured data rather than a free-form essay. Each suggested relationship should include source course, destination course, relationship type, explanation, and uncertainty information. The backend must validate the response format before using it.

A model-reported confidence value is not proof of correctness and should not replace evidence, review, or validation.

4.3 Validation rules: the control mechanism

- Check that both ends of every proposed relationship are among the user's selected courses.
- Normalize aliases and flag names that could mean more than one thing.
- Compare AI suggestions with the knowledge base and flag disagreements for review.
- Distinguish formal prerequisites from recommendations and optional supporting knowledge.
- Detect suspicious or contradictory dependencies, but do not silently delete an edge just to force an ordering.
- Preserve explanations and provenance so the team can tell why a relationship was included.

4.4 Handling missing or conflicting information

If a course is unknown, ask for clarification or allow a carefully labeled AI suggestion. If sources disagree, show the disagreement or mark the relationship uncertain. Never present an unverified suggestion as an official curriculum requirement.

5. Representing Prerequisites as a Graph

Once relationships have been validated, the DAA engine needs a formal representation. We will use a directed graph.

Graph term	Meaning in this project
Node / vertex	A course, such as Python Programming or Machine Learning.
Directed edge	A relationship with a direction. A → B means A is recommended or required before B.
Incoming edge	A relationship pointing into a course; it represents a dependency the ordering must respect.
Outgoing edge	A relationship pointing from a course to another course that may depend on it.
Cycle	A path of directed relationships that eventually returns to its starting course.

Adjacency list

An adjacency list stores, for each course, the courses directly reachable from it. It is compact and convenient for a sparse dependency graph.

```
graph = {
  "Python": ["DSA", "Machine Learning"],
  "DSA": ["Artificial Intelligence"],
  "Artificial Intelligence": ["Machine Learning"],
  "Machine Learning": ["Deep Learning"],
  "Deep Learning": []
}
```

This graph is only an example for explaining data representation. The team must review whether each relationship is appropriate for the intended course descriptions and learning goal.

Required, recommended, and optional are different

Relationship type	Meaning	How the app should present it
Required	A formal requirement for a specific course or curriculum, supported by a source.	Show as required only when verified.
Recommended	The earlier course is likely to make the later course easier to understand.	Use for the default learning path when justified.
Optional	Useful background, but not necessary to begin the next course.	Show as supplementary rather than blocking progress.

6. DAA Concepts and Their Role

6.1 Directed graphs and graph representation

The graph models dependencies. Direction matters: A → B does not mean the same thing as B → A. The graph is built from validated relationships, not directly from raw AI text.

6.2 Topological sorting

Topological sorting produces an order of nodes in a directed graph such that every edge A → B places A before B. It works only when the graph is a directed acyclic graph (DAG). In our project, it generates a learning order that respects accepted prerequisite relationships.

Kahn's Algorithm — simple explanation

- Count incoming edges for every course.
- Put all courses with zero incoming edges into a queue.
- Remove one course from the queue and add it to the output order.
- Remove that course's outgoing edges; courses whose incoming-edge count becomes zero can enter the queue.
- Repeat. If every node is output, we have a topological order. If nodes remain, the graph contains a cycle.

6.3 Depth-First Search (DFS)

DFS explores a path as far as it can before returning to explore other paths. We can use DFS for graph traversal, cycle detection, and a second implementation of topological sorting. Comparing DFS-based sorting with Kahn's Algorithm gives the project a useful algorithm-comparison component.

6.4 Cycle detection

A cycle represents a structural conflict. For example, A → B, B → C, and C → A cannot all be respected in a linear topological order. The application should report the conflict and allow relationships to be reviewed. The graph algorithm identifies the cycle; it does not know which educational statement is wrong.

6.5 Time complexity and performance analysis

Let V be the number of courses (vertices) and E be the number of relationships (edges). With an adjacency-list representation, Kahn's Algorithm and DFS-based topological sorting each run in O(V + E) time. This means the work grows in proportion to the number of nodes plus edges, assuming a standard implementation.

Concept	Planned use	Expected complexity
Graph construction	Build nodes and adjacency lists from validated relationships.	O(V + E), depending on implementation
Kahn's topological sort	Generate a valid order and detect a cycle if nodes remain.	O(V + E)
DFS traversal / topological sort	Explore dependencies and produce an alternative ordering.	O(V + E)
Cycle detection	Flag cyclic dependency structures.	O(V + E) with standard graph methods

Measure graph algorithms separately from the AI API call. Network latency can dominate total response time and make the graph algorithms appear slower than they really are.

7. Proposed Technology Stack and System Modules

Technology	Role
React + TypeScript	Builds the interactive frontend and organizes UI logic.
Tailwind CSS	Provides styling and responsive layout.
React Flow	Displays and supports interaction with the prerequisite graph.
Python + FastAPI	Receives frontend requests and coordinates backend processing.
LLM API (provider TBD)	Suggests relationships and explanations from selected courses and context.
JSON knowledge base	Stores course names, aliases, descriptions, and reviewed relationships.
Python graph engine	Implements graph construction, Kahn's Algorithm, DFS, and cycle checks.
Plotly	Creates charts for the algorithm-analysis dashboard.
Pytest	Runs automated tests for algorithms and backend behavior.

Suggested backend modules

Module	Responsibility
course_normalizer.py	Normalize names, resolve aliases, identify duplicates and ambiguous names.
knowledge_base.py	Load and query the JSON course information and reviewed relationships.
prerequisite_analyzer.py	Call the AI API and parse structured relationship suggestions.
relationship_validator.py	Validate endpoints, types, uncertainty, provenance, and conflicts.
graph_builder.py	Build the graph and adjacency list from accepted relationships.
graph_algorithms.py	Implement Kahn's Algorithm, DFS, topological sorting, and cycle detection.
analysis_service.py	Coordinate stages and return a structured result to the frontend.
main.py	Create the FastAPI application and API endpoints.

No database is required at the beginning

A JSON knowledge base is enough for a prototype. A database can be added later if the team needs saved user plans, accounts, edits, history, or a larger searchable catalogue. Avoid adding infrastructure before it solves a real need.

8. Example Walkthrough: From Input to Output

Assume a student enters the following courses and selects the goal AI Engineer:

Python Programming • Linear Algebra • Machine Learning • Deep Learning

Stage A: Normalize the input

The backend removes duplicates and resolves aliases. For example, 'ML' may be normalized to 'Machine Learning' if the context makes the meaning clear. Ambiguous names should be clarified rather than guessed silently.

Stage B: Retrieve known information

The knowledge base supplies stored descriptions and reviewed relationships. It may already contain useful information about programming, mathematics, Machine Learning, and Deep Learning.

Stage C: Ask the AI for additional suggestions

The AI proposes relationships among selected courses and provides explanations. For illustration, it might suggest Python Programming → Machine Learning, Linear Algebra → Machine Learning, and Machine Learning → Deep Learning. The exact suggestions must be reviewed and validated.

Stage D: Validate and build the graph

The backend checks that all relationship endpoints are selected courses, preserves the relationship type and reason, and flags conflicts. Accepted relationships become directed edges in the graph.

Stage E: Run the DAA engine

Kahn's Algorithm generates an order that respects accepted edges. DFS can produce another valid order. If a cycle is present, the application reports that no topological ordering exists until the conflict is resolved.

Stage F: Display the result

- Interactive graph with arrows showing accepted relationships.
- Recommended learning order with explanations.
- Labels distinguishing required, recommended, and optional knowledge.
- Warnings for unknown courses, uncertain relationships, or cycles.
- Algorithm results and performance measurements.

A valid output could place Python Programming and Linear Algebra before Machine Learning, and Machine Learning before Deep Learning. The relative order of Python and Linear Algebra may vary if there is no edge connecting them. This is normal: a graph can have more than one valid topological ordering.

9. Evaluation, Testing, and Success Criteria

Evaluate educational inference and graph algorithms separately.

9.1 Educational recommendation tests

- Prepare a reviewed set of test cases containing different course combinations and learning goals.
- Include programming, DSA, mathematics for ML, ML/Deep Learning, databases/data engineering, unrelated courses, and ambiguous names.
- Review whether each suggested relationship has a defensible direction and an appropriate relationship type.
- Track unsupported suggestions, missing relationships, and uncertain cases.
- If enough labeled examples are available, calculate precision, recall, and F1 for relationship inference; do not report these metrics without a reviewed test set.

9.2 DAA correctness tests

Test case	Expected behavior
Empty graph	Return a clear empty-input result without crashing.
One course, no edges	Return that course as a valid order.
Several disconnected courses	Return all courses; relative order may vary.
A → B → C	Place A before B and B before C.
A → B, A → C	Place A before both B and C; B and C may appear in either order.
A → B → C → A	Detect a cycle and report that no topological order exists.
Unknown edge endpoint	Reject or flag the relationship before graph processing.

9.3 Performance tests

Create synthetic graphs of increasing sizes, such as 10, 50, 100, 500, and 1,000 nodes with different edge counts. Run Kahn's Algorithm and DFS-based topological sorting repeatedly and record their execution times. These are planned test sizes, not results.

Success criteria

- The graph engine returns a valid order for every acyclic test graph.
- Cycle detection correctly identifies cyclic test graphs.
- The application distinguishes graph conflicts from educationally incorrect relationships.
- Every accepted relationship has a type and an explanation or provenance status.
- The UI communicates uncertainty and does not label recommendations as official requirements without evidence.
- Performance measurements isolate graph algorithms from AI API latency.

10. Limitations, Risks, and Responsible Design

Risk	Why it matters	Mitigation
AI can be wrong	A fluent explanation can still be inaccurate.	Use reviewed knowledge, validation, provenance, and user review.
Course names are ambiguous	An abbreviation may refer to different subjects.	Normalize known aliases and ask for clarification when needed.
Curricula differ	A prerequisite in one course may be optional in another.	Use relationship types and allow goal/context to affect recommendations.
Cycles occur	The graph cannot be topologically sorted while a cycle remains.	Detect and report the conflict; review relationships instead of silently deleting edges.
Valid order may not be optimal	Topological sorting enforces dependencies but does not optimize workload, time, or ability.	Describe the result as a valid recommended order, not a guaranteed optimal plan.
AI API latency or limits	External calls may be slow, unavailable, or costly.	Add timeouts, error handling, limited retries, and clear failure messages.
API key exposure	A key in frontend code can be stolen.	Keep the key on the backend in environment variables; never expose it in the frontend.
Unverified confidence score	A model-reported score may be misleading.	Treat it as a weak signal, not proof; prefer evidence and explicit uncertainty.

Critical design rule

The graph algorithm answers: 'Can these relationships be arranged into a valid order?' It does not answer: 'Are these relationships educationally correct?' Those are different questions and must be tested separately.

11. Step-by-Step Implementation Roadmap

Build one layer at a time. First make the algorithmic core dependable, then connect the knowledge base and AI, and only then complete the interface and dashboard.

Stage	Build	Definition of done
1. Graph algorithm engine	Adjacency list, Kahn's Algorithm, DFS, topological sorting, cycle detection.	Algorithms pass tests on simple, disconnected, cyclic, and larger graphs.
2. Knowledge base	Initial JSON catalogue, aliases, course descriptions, reviewed relationships.	Backend finds courses and retrieves known relationships consistently.
3. AI integration	Prompt design, API call, structured response parsing, error handling.	Suggestions use a predictable schema; invalid responses are handled.
4. Validation layer	Check endpoints, types, uncertainty, conflicts, and provenance.	Questionable suggestions are flagged rather than silently trusted.
5. FastAPI integration	Endpoint coordinating input processing through graph output.	A test request returns structured relationships, graph diagnostics, and ordering.
6. Frontend	Course form, learning goal, result cards, graph, learning order.	A user can submit courses and understand the result.
7. DAA dashboard	Algorithm outputs, graph statistics, timing comparisons, charts.	Dashboard shows measured results and test conditions.
8. Evaluation and report	Test cases, result tables, complexity explanation, limitations.	Team can explain what works and what remains uncertain.

Suggested team practice

- Keep graph algorithms independent of the AI API so they can be tested without network access.
- Agree on one relationship data format before team members implement separate modules.
- Use version control and small changes; test after each stage.
- Keep example data clearly marked as illustrative until reviewed.
- Record actual benchmark results rather than inventing measured performance.

12. Key Terms and Viva-Ready Summary

Term	Plain-language meaning
Knowledge base	Stored course information and reviewed relationships the application can look up.
API	A way for our backend to request an answer from an external AI model.
LLM	An existing AI model that can interpret course names and propose relationships in structured form.
Validation	Checks that a proposed relationship is well-formed, relevant, and appropriately labeled.
Directed graph	A set of nodes connected by arrows that have a direction.
Vertex / node	A course in our application.
Edge	A directed relationship between two courses.
Adjacency list	A way to store each node's directly connected nodes.
Topological sorting	An ordering in which every prerequisite edge points from an earlier course to a later course.
Kahn's Algorithm	A queue-based method for topological sorting that can also reveal a cycle.
DFS	Depth-First Search; a graph traversal method that explores a path before backtracking.
Cycle	A directed path that eventually returns to its starting node.
Time complexity	A description of how algorithm work grows as input size increases.
O(V + E)	For standard adjacency-list implementations, work proportional to vertices plus edges.

A short explanation for the viva

Our project is an AI-assisted prerequisite learning-path generator. The user enters courses and a learning goal. The backend normalizes course names, consults a curated knowledge base, and asks an AI API to suggest additional relationships. A validation layer checks suggestions and flags uncertain or conflicting edges. We represent accepted relationships as a directed graph and use Kahn's Algorithm for topological sorting, DFS for graph traversal and an alternative topological sort, and cycle detection to identify conflicts. We analyze algorithms using time complexity and measured performance on test graphs. The AI helps propose educational relationships; the DAA engine organizes and analyzes the graph.

Final architecture decision

This is the agreed initial architecture for implementation. We will not train a custom ML model in the first version. We will first implement and test graph algorithms, then build the knowledge base and AI inference layer, connect the backend, create the frontend, and finish with evaluation and documentation.